

Multiple Classes Inheriting from the Same Base Class

Multiple different classes can inherit from the same base class. You can build a very general base class, then construct any number of subclasses that inherit from it. [Figure 10-6](#) is a representation of this relationship.

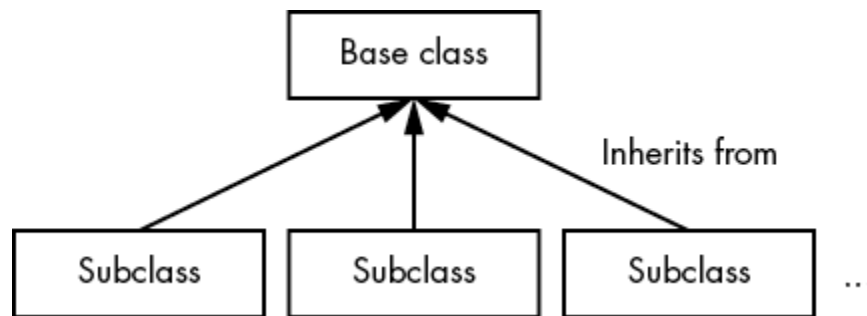


Figure 10-6: Three or more different subclasses inheriting from a common base class

Each of the different subclasses can then be a variant (a more specific version) of the generic base class. Each subclass can override any methods of the base class that it wants to or needs to, independent of any other subclass.

Let's walk through an example using the Shapes program from Chapter 9 that created and drew circles, squares, and triangles. The code also allowed the user to click on any shape in the window to see the area of that shape.

The program was implemented with three different shape classes: `Circle`, `Square`, and `Triangle`. If we look back at those three classes, we find that each one has this exact same method:

```
def getType(self):  
    return self.shapeType
```

Further, looking at the `__init__()` methods of the three classes, we find that there is some common code that remembers the window, chooses a random color, and chooses a random location:

```
self.window = window  
self.color = random.choice((RED, GREEN, BLUE))
```
